Department of Computer Science, University College London

COMP0227 Lab 05: Objects over Time

3rd November, 2026

1 Scope and Purpose

In Lab 04 we found the objects in single images and in the frames of a sequence. A detector has no memory: the chair it finds in this frame and the chair it found a second ago are two detections, not one chair. A *tracker* joins detections over time into *tracks*. It predicts where each tracked object will be in the next frame, matches the new detections to the predictions, and updates each track with the detection it was matched to. Each track carries an *identity*, a number which should stay the same while its object is in view.

pySLAM uses two trackers from the same package as its detector, and SORT, the simplest tracker of the same kind, to compare them with. This lab has two main purposes. The first is to run them on your machines, on sequences with people who walk about and a camera that moves. The second is to carry out a small investigation into when an identity holds, and into what tracks add to a map. Specifically:

1. When does an object keep its identity, and when does it lose it?
2. How does the tracker predict where a box will be in the next frame?
3. When the tracks are lifted into the map with the poses from SLAM, do the things which stay still become one object each?

You will first run the trackers and watch the identities. You will then write the tracker's prediction step yourself. Finally you will lift the tracks into the map, with and without the people masked out of SLAM, and compare the objects with those of Lab 04's single detections.

This lab assumes that you have completed Labs 00 to 04, and that pySLAM is installed and working. As before, run all the commands from the `pyslam` folder.

Each tracking run takes about a minute on a GPU and a few minutes on a CPU; each SLAM run of Part 3 takes about a minute. The members of your group can divide the runs among themselves: the table of Activity 2 needs four runs, and Part 3 needs the tracks of one run, which one member saves and shares (7 MB).

2 Setting Up

Do this before the lab: the downloads take a while.

First bring pySLAM up to the version for this lab, as the CHANGELOG describes. From the `COMP0227_26-27` **folder:**

```
git pull
git submodule update --init --depth 1 pyslam
cd pyslam
pixi run build
```

The commands are the same on Linux, macOS and native Windows. `build` **only does what has changed, and nothing you installed before is lost. The version for this lab is** `comp0227-2026-lab05`**.**

The trackers are in the `semantics` **level, with the detector of Lab 04. Lab 04's Setting Up (**`models-detection`**) downloaded the detector's weights into the folder** `data/models/`**. The level gains one small package for the trackers (1.4 MB), which pixi installs by itself the first time you use the level after the update.**

- **On Linux without an NVIDIA GPU (in WSL2 too), use** `-e semantics-cpu` **instead of** `-e semantics` **everywhere, and add** `-e default-cpu` **to every** `pixi run` **command of this sheet which has no** `-e`**, straight after** `pixi run`**, as in Lab 04.**
- **If disk space is short,** `pixi clean cache` **deletes the copy of the downloaded packages which pixi keeps.**

The lab uses two sequences of the TUM RGB-D benchmark, each a hand-held camera in an office with two people walking about:

- `rgbd_dataset_freiburg3_walking_xyz` **(**`walking_xyz` **below; a 0.53 GB download): the camera moves slowly. It was the extra activity of Lab 02, so you may have it already.**
- `rgbd_dataset_freiburg3_walking_halfsphere` **(**`walking_halfsphere` **below; 0.64 GB): the camera swings over half a sphere.**

Download and unpack both. Part 3 runs SLAM with the depth images, so, as in Lab 03, write the file which pairs each colour image with its depth image in each folder:

```
S=path/to/rgbd_dataset_freiburg3_walking_xyz
pixi run python scripts/associate.py $S/rgb.txt $S/depth.txt > $S/associations.txt
H=path/to/rgbd_dataset_freiburg3_walking_halfsphere
pixi run python scripts/associate.py $H/rgb.txt $H/depth.txt > $H/associations.txt
```

`S` **and** `H` **are shorthands for the two folders, and the commands below use them too; they last only until the terminal is closed, so set them again in each new terminal. The two files list 827 frames (of** `walking_xyz`**'s 859 colour images) and 1021 frames (of** `walking_halfsphere`**'s 1067). The tracker uses every colour image; SLAM uses the frames of the list.**

On Native Windows

Native Windows runs every command of this sheet as it is written, on the CPU. The tracking commands took about 0.07 s per frame on a desktop CPU, so about a minute for `walking_xyz`**; on a slower laptop add** `--step 2`**, as on any CPU.** `-e default-win`**, from earlier sheets, still works for**

the commands without `-e`. The PowerShell notes of Lab 02 apply here too: set a folder with `$S = "C:\path\to\rgbd_dataset_freiburg3_walking_xyz"`, write an associations file with `| Out-File -Encoding ascii` instead of `>`, and type each command on one line, without the `\` at the ends of the lines.

3 The Methods

Table 1 lists the trackers. The names are the ones pySLAM uses. `SORT` is there for comparison: it has one stage and no Lost tracks.

Name	Matches by	The camera's motion	The box state
`BYTETRACK`	the overlap (IoU) of each track's predicted box with the boxes, in two stages: the high-score boxes first, at a cost of $1 - \text{IoU} \times \text{score}$, then the low-score boxes, at $1 - \text{IoU}$	ignored	centre x and y, aspect ratio, height
`BOTSORT`	the same two stages, after a warp fitted to the image has moved every predicted box	estimated from the image	centre x and y, width, height
`SORT`	the overlap of each track's predicted box with the high-score boxes only, in one stage, at a cost of IoU; a pair overlapping less than 0.3 is not matched	ignored	centre x and y, area, aspect ratio

Table 1: The trackers of this lab, ByteTrack, BoT-SORT and SORT. Each predicts each track's box with a Kalman filter on its box state: ByteTrack and BoT-SORT with the state's four velocities, SORT with the velocities of the centre and the area. All three run on the detections of `YOLO` from Lab 04. BoT-SORT can also compare the objects' appearance, which is off in its settings file.

A good tracker keeps an object's identity through a short occlusion, and does not hand it to a different object. A track which finds no detection is *Lost*: it is only predicted, and not reported, and it is kept for a while in case its object comes back; how long is set in the tracker's settings file. SORT reports a new track only after it has been matched in 3 frames in a row, and deletes a track at its second frame in a row without a box. The settings files are in `settings/trackers/`, `bytetrack.yaml`, `botsort.yaml` and `sort.yaml`, and you may edit them. The full list of trackers, with the detectors which can feed them, is printed by:

```
pixi run -e semantics object-tracking --list
```

4 Part 1: Watch the Identities

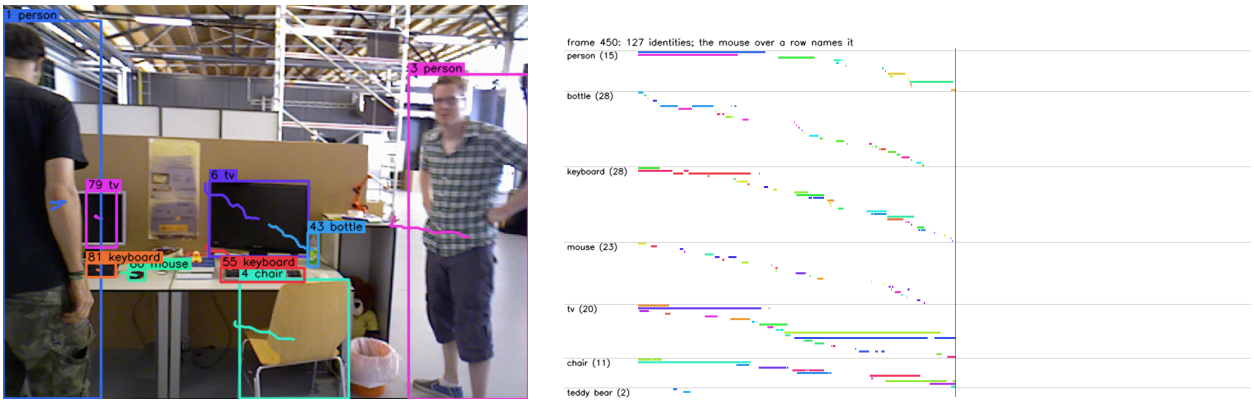
Activity 1: Track a Sequence

Run:

```
pixi run -e semantics object-tracking --tracker BYTETRACK --tum $S \
    --settings settings/TUM3.yaml
```

This detects the objects of every frame with `YOLO` and tracks them. The 859 frames take under a minute on a GPU. On a CPU they take one to three minutes: there, add `--step 2` to the tracking commands of Parts 1 and 2, which tracks every second frame. Two windows open (Figure 1):

- `Tracks`: **the frame, with every tracked box, its identity and class, and a trail of where its centre has been over the last 30 frames. A box drawn with a thin line, labelled "(2nd stage)", was matched in the second stage (Activity 3).**
- `Timeline`: **one row per identity, grouped by class with the people first, filled in the frames where the identity was seen, and a red line at the current frame. A track that breaks, or a number that changes, shows at a glance. With the mouse over a row, the window's title names that identity and the frames where it was seen.**



(a) `Tracks`, at frame 100 (b) `Timeline`, at frame 450

Figure 1: The two windows of Activity 1 on `walking_xyz`, from our run. The `Timeline` is shown half way through the 859 frames; yours at the end gives the counts of Table 2.

Press `p` to pause and `q` or Esc to stop. At the end of the sequence the windows stay open until you press `q` or Esc. The run ends with a one-line summary in the terminal. Here is the summary of our run on another sequence, the KITTI 06 street video of the worked example after Activity 3:

```
Summary: BYTETRACK on YOLO, kitti06/video_color.mp4 (1101 frames): 0.019 s
per frame on mps (the tracker alone 0.5 ms); 239 identities, 2 of them people;
longest track 96 frames (car, identity 671); 125 tracks shorter than 10
frames; 157 of 3211 boxes matched in the second stage
```

Its parts mean the following:

- *0.019 s per frame*: **the whole frame, detector and tracker together;** *the tracker alone 0.5 ms* **is the tracker's part of it.**
- *239 identities, 2 of them people*: **the numbers handed out over the whole run, and how many of them went to boxes of the class `person`. An identity's class is the class most of its boxes carry.**
- *longest track 96 frames (car, identity 671)*: **the identity seen in the most frames. At 10 frames per second, 96 frames is about 10 of the video's 110 seconds.**
- *125 tracks shorter than 10 frames*: **identities seen in fewer than 10 frames, here one second. `walking_xyz` has 30 frames per second, so there 10 frames are a third of a second.**
- *157 of 3211 boxes matched in the second stage*: **of all the boxes the tracker reported, those matched in its second stage (Activity 3), here 4.9%.**

Write the numbers from your own summary for `walking_xyz` into the first row of Table 2. COCO, the detector's set of classes, calls a computer monitor `tv`.

- **Two people walk about. How many identities did they get between them? Find in the `Timeline` where a person's number changes, and pause the run there. What happened just before: did the two cross, did one leave the picture, did the detector miss a frame?**
- **Look at the monitors and the chairs, which do not move. Do they keep one identity each? What breaks their tracks?**
- **The `Timeline` has many tracks of a few frames. Which classes are they? What makes a track that short?**
- **The time per frame has two parts, the detector and the tracker. Which takes most of it?**

Activity 2: Two Trackers, and a Camera that Moves

Run the same sequence with `BOTSORT`, and then both trackers on `walking_halfsphere`:

```
pixi run -e semantics object-tracking --tracker BOTSORT --tum $S \
    --settings settings/TUM3.yaml
pixi run -e semantics object-tracking --tracker BYTETRACK --tum $H \
    --settings settings/TUM3.yaml
pixi run -e semantics object-tracking --tracker BOTSORT --tum $H \
    --settings settings/TUM3.yaml
```

Each takes about as long as Activity 1. Split the runs in your group, and fill in rows 2 to 4 of Table 2 from the summary lines. While a run goes, watch its `Timeline` grow, and compare it with Activity 1's.

	Identities	People	Longest	Short	s per frame	Tracker (ms)
1. `BYTETRACK`, `walking_xyz`						
2. `BOTSORT`, `walking_xyz`						
3. `BYTETRACK`, **halfsphere**						
4. `BOTSORT`, **halfsphere**						
5. `BYTETRACK`, **2nd stage off**						
6. `SORT`, `walking_xyz`						

Table 2: The runs of Activities 1 to 3, from their summary lines. Longest: the longest track, in frames, and its class. Short: tracks shorter than 10 frames. s per frame: the time per frame, and Tracker: the tracker's share of it. Rows 5 and 6 are on `walking_xyz`.

- **Which tracker gave fewer identities on the same detections? To the people? Where in the `Timeline` does the difference come from?**

- **Which tracker is slower, and by how much? Is the difference in the detector or in the tracker? On which computer would it matter?**
- **On `walking_halfsphere` the camera swings about. A tracker predicts each box from its own velocity in the image. What happens to every box at once when the camera turns, and what does that do to the overlap between the predicted boxes and the detections?**
- **Did both trackers get worse on `walking_halfsphere`? Which got worse by more: count the extra identities, and also compare the two as ratios. Does compensating the camera's motion help as much as you expected?**

Activity 3: The Low-Score Boxes

The second stage takes the boxes that the detector was not sure about, the *low-score* boxes (scores from 0.1 to 0.25). It offers them to the tracks that were tracked in the previous frame and found no box in the first stage, at a cost of $1 - \mathrm{IoU}$, without the score: a person half hidden behind a chair is still there, with a low score. A low-score box that no track takes is dropped: it never starts a track. `--second-stage off` turns the second stage off, by raising the lower threshold to the upper one so that no box is left for it:

```
pixi run -e semantics object-tracking --tracker BYTETRACK --second-stage off \
    --tum $S --settings settings/TUM3.yaml
```

Write its line into row 5 of Table 2.

- **How many identities now? The difference may be small: the summary line of Activity 1 said how many boxes the second stage matched at all. Find in the two `Timeline`s a place where the runs differ, and pause there. What happened to the track that the second stage would have kept?**
- **The thresholds are in `settings/trackers/bytetrack.yaml`, and each run prints the values it uses. `track_buffer` is the number of frames for which a Lost track is kept, in case its object comes back. Change it from 30 to 5, and run Activity 1's command again. Which tracks break, and where? What happens to the longest track? Then set it back to 30.**

SORT goes further than `--second-stage off`: it gets only the high-score boxes (0.25 or more), matches them once, keeps no Lost tracks, and holds a new track back until it has been matched in 3 frames in a row. Run it on the same sequence:

```
pixi run -e semantics object-tracking --tracker SORT --tum $S \
    --settings settings/TUM3.yaml
```

Write its line into row 6 of Table 2. Figure 2 is the `Timeline` of our run: compare it with yours, and with ByteTrack's in Figure 1. Its summary ends with two counts of its own:

```
...; no second stage (boxes of 0.25 or more); N tracks started, M matched
boxes held back while their track was on probation
```

Your counts: _____ tracks started, _____ matched boxes held back.

- **Compare SORT's line with ByteTrack's of Activity 1 and with `--second-stage off`. Which gives the people fewer identities? Is fewer identities better here: look in the `Timeline` at the rows of the two people, and at how long each person goes unreported.**
- **Which class has the longest track for ByteTrack, and which for SORT? If they differ, what ends that track in one tracker but not in the other?**

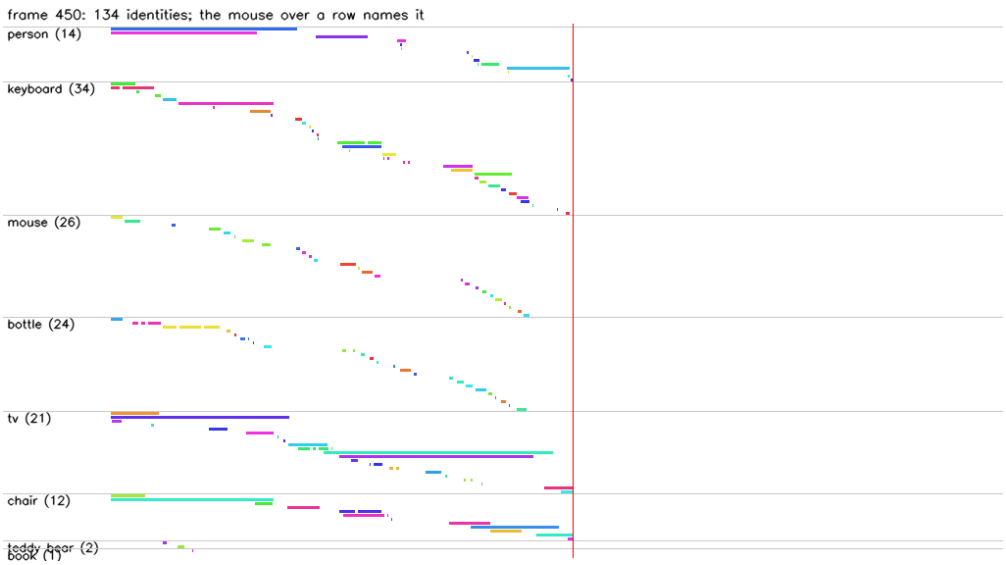


Figure 2: The `Timeline` of `SORT` on `walking_xyz`, at frame 450, as Figure 1. The rows are grouped by class, with the number of identities of each class so far.

- **Compare the tracks SORT started with the identities it reported. What happened to the other tracks, and what does the probation cost when an object first appears?**
- **In `settings/trackers/sort.yaml`, `max_age` is the number of frames a track may go without a box before it is deleted. Change it from 1 to 30, so that it keeps its tracks about as long as ByteTrack's `track_buffer`, and run again. Which counts move towards ByteTrack's? Then set it back to 1.**

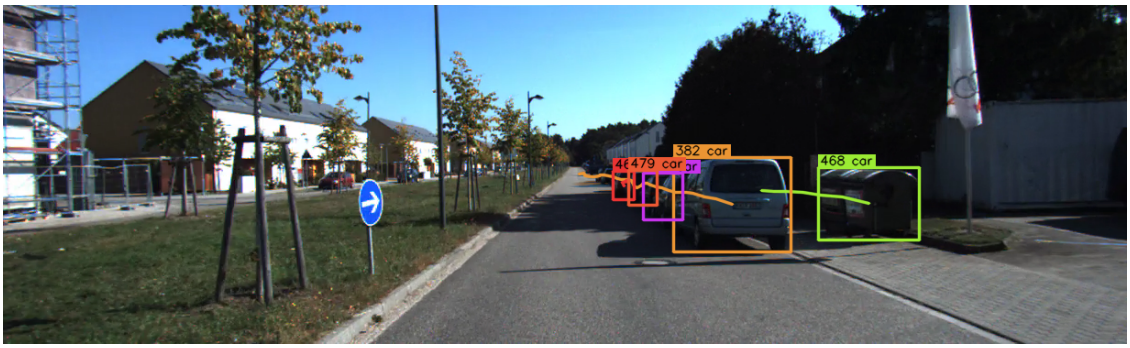
Worked Example: The Same Trackers on a Street

This example needs no run: read it, and compare it with your own runs. We tracked the KITTI 06 video of Lab 00 (1101 frames of a car driving through a suburb, at 10 frames per second) with `SORT` and with `BYTETRACK`, with Activity 1's command and `--video data/videos/kitti06/video_color.mp4 --settings settings/KITTI04-12.yaml` in place of `--tum` and its settings. Activity 9 runs it if you want to try. The two summaries, side by side:

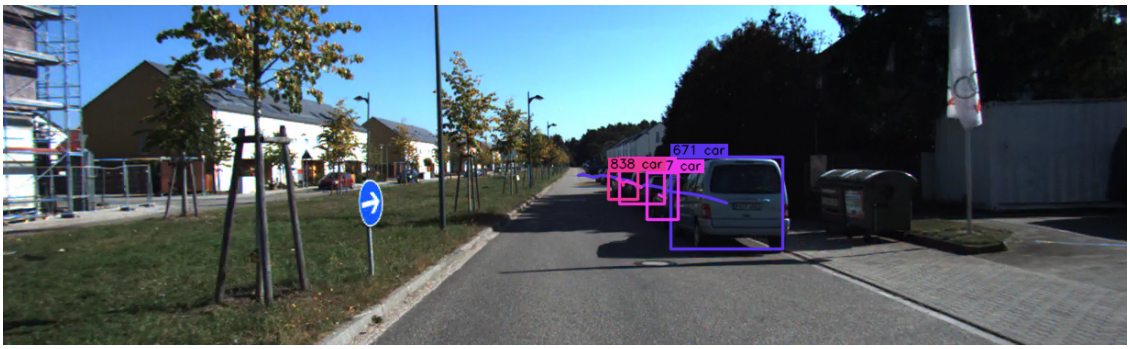
	SORT	BYTETRACK
Identities (people)	**327 (5)**	**239 (2)**
Longest track, frames	**92 (a car)**	**96 (a car)**
Tracks shorter than 10 frames	**222**	**125**
Tracks started; matched boxes held back	**1211; 2034**	**–**
Boxes matched in the second stage	**–**	**157 of 3211**
The tracker alone, ms per frame	**0.2**	**0.5**

Table 3: `SORT` and `BYTETRACK` on KITTI 06 (our runs, the same detections; 0.019 s per frame for both).

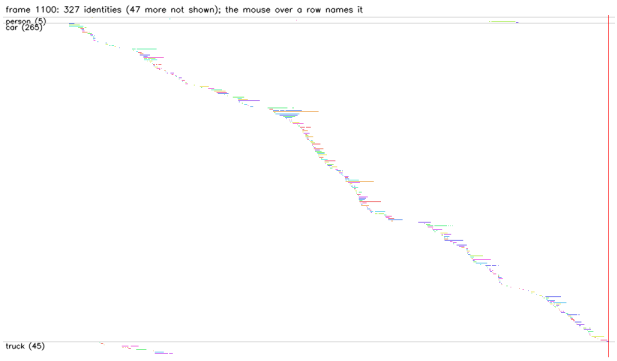
In both `Timeline`s (Figure 3) the rows of the cars run down in a diagonal: the car drives past the parked cars one after another, and each is seen for a few seconds and then leaves the picture for good. Unlike `walking_xyz`, a track here mostly ends because its object has gone, not because the tracker lost



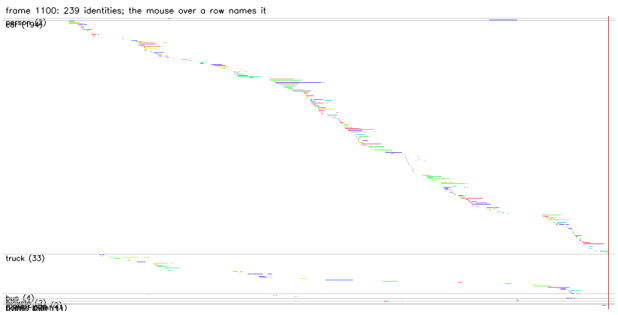
(a) SORT



(b) BYTETRACK



(c) SORT, Timeline at the end



(d) BYTETRACK, Timeline at the end

Figure 3: The Tracks window at frame 500 of KITTI 06, and the two Timelines. KITTI images: Geiger, Lenz and Urtasun, the KITTI Vision Benchmark Suite, CC BY-NC-SA 3.0.

it. Even so, SORT has almost twice as many short tracks as BYTETRACK. At 10 frames per second a box moves far between frames, so a track's predicted box often overlaps its next box little.

- **Compare the ratios of the two trackers' identities here and on walking_xyz (Activities 1 and 3). On which sequence does SORT fall further behind? What about the street makes it harder for a tracker with no Lost tracks and no low-score boxes?**
- **At frame 500, SORT reports a box on the right, labelled car (identity 468), which BYTETRACK does not report. Look at the image: what is it? Which of the two trackers' rules for starting a track let it through?**

5 Part 2: Inside the Tracker

Activity 4: Write the Prediction

Between two frames, each track's box is predicted with a Kalman filter on its state. The state is a vector of eight numbers: the box state of Table 1 and its four velocities, in pixels per frame. The model is constant velocity, so each of the first four numbers moves by its velocity times the time step, and the velocities stay as they are. For the centre x alone, with $x = 100$ pixels, a velocity of 3 pixels per frame and a time step of one frame, the box is predicted at 103 pixels.

Open the file `pyslam/semantics/track_utils.py`**. It contains the function**

```
def predict_box(state, covariance, dt=1.0):
```

which, as supplied, returns `None`**. Its docstring says what it is given and what it must return. In short, it gets the state of a track (a float array of shape (8,)), its 8×8 covariance, and the time step** `dt` **in frames. It must return the predicted state and the predicted covariance. The file also gives** `process_noise(state)`**, which returns the process noise Q that the tracker uses for that state. The update step, which corrects a track with the detection it was matched to, is the tracker's: you are writing its first half. Write it; you need about eight lines:**

- **build the 8×8 transition matrix F: the identity, with** `dt` **where row i meets column $i + 4$, for i from 0 to 3;**
- **return F** `@ state` **and F** `@ covariance @ F.T + Q`**.**

Useful functions: `np.eye`**, indexing such as** `F[i, i + 4] = dt`**, the matrix product** `@`**, and** `.T`**. Then run the tracker with your prediction:**

```
pixi run -e semantics object-tracking --tracker BYTETRACK --predict own \
    --tum $S --settings settings/TUM3.yaml
```

The `Tracks` **window now also draws each track's predicted box, dashed. If your function still returns** `None`**, the run says so once and uses the tracker's own prediction. The summary line ends by saying which prediction was used, and how far your predicted boxes were from the tracker's own:**

```
...; prediction: predict_box(), 0.00 px at most from the tracker's own
```

- **Do your identities match Activity 1's? If they do not, how far off were your boxes, and which part of F would put them there?**
- **Set** `dt` **to 2 (change the default in the function's first line): the tracker now thinks that a frame was skipped. What happens to the dashed boxes when a person turns round? How far from the tracker's own prediction do they get, and how many identities are there now?**
- **Set** `dt` **back to 1, and multiply Q by ten in your function. The summary still says 0.00 px: why do the predicted boxes not move? Q enters the covariance, and the covariance decides how far the update moves a track towards its detection. Does the filter now trust its prediction more, or the detections? What happens to the number of identities?**
- **These trackers match the predicted boxes to the detections by their overlap, times the score in the first stage, and use the predicted covariance only in the update. Where else could a tracker use it?**

6 Part 3: Tracks into the Map

A track is a sequence of boxes in the image. With the camera's pose at each frame from SLAM, and the depth of the frame, each box becomes a point in the map, as each detection did in Lab 04. A track then becomes a path, and a thing which stays still should collapse to a point.

Activity 5: Lift the Tracks

Save the tracks of `walking_xyz`**, then run SLAM with them:**

```
pixi run -e semantics object-tracking --tracker BOTSORT --tum $S \
    --settings settings/TUM3.yaml --save tracks/walking_xyz
pixi run slam --tum $S --settings settings/TUM3.yaml --sensor rgbd \
    --tracks tracks/walking_xyz
```

`--save` **writes the boxes of every frame, with their identities, classes and masks, into the folder** `tracks/walking_xyz`**.** `--tracks` **reads them. For each frame which SLAM tracks, each box of that frame is lifted as a detection was in Lab 04: the centroid of its mask, at the median depth under the mask, is moved into the map with the frame's own pose. SLAM runs at the camera's rate, so the 827 frames take about a minute: watch the paths grow while it runs. In the** `Map Viewer`**, each identity's points are joined into a path, in the identity's colour in the** `Tracks` **window. At the end of the sequence, pySLAM turns the tracks into objects, and the** `Map Viewer` **draws them as large points, one colour per class:**

- **A track is *static* if 90% of its lifted points lie within 0.30 m of their median.**
- **The static tracks are then merged, the longest first (by their number of lifted boxes): each joins the nearest object of its class whose centre is at most 0.35 m away, and otherwise starts a new object. An object's centre is the mean of its tracks' centres, weighted by their boxes, and a track's class is the class that most of its boxes carry. Lab 04 merged detections with the same test, but in order of score, each with a weight of one.**
- **The two distances are the options** `--static-radius` **and** `--merge-radius`**.**
- **When the run ends, the boxes are lifted again with the final trajectory, which bundle adjustment has refined, and merged again.**

Press `q` **or Esc in the** `Camera` **window to end the run. The terminal then gives the trajectory error, how many boxes were lifted, and the objects, one line each:**

```
Tracks: 5812 boxes lifted in 825 frames, with the depth images; 399 boxes had
too little depth
Objects: 27 from 64 static tracks: chair x7, keyboard x5, tv x5, bottle x3,
mouse x3, person x3, teddy bear x1; moving: 16 tracks (...)
  tv: centre (-0.53, -0.09, 3.05) m; 8 tracks (110, 251, 422, 353, 5, 73, 53,
  49), 696 boxes; 90 % of its points within 0.19 m
  ...
ATE rmse 0.8719 over the tracked frames: 826 of 827 frames tracked (100%)
```

Your numbers will be different. Earlier versions of pySLAM now and then froze a run: the frames stopped advancing, and the run used no CPU. This version fixes that. If a run still freezes, Ctrl-C does not stop it: close the terminal window (on Linux and macOS, Ctrl-\ also works), open a new one, set `S` **again, run the command again, and tell us.**

- **Follow one person's path. Where does it jump? Do the jumps line up with the changes of identity that you saw in Part 1?**
- **Do the monitors and the chairs collapse to a point? Each object's line says within what distance 90% of its points lie (Lab 04's spread was the largest distance instead). How large is the cloud each one leaves? What could it be made of: the noise of the depth, the box moving on the object, or the pose?**
- **Look at the trajectory error. Lab 02 masked the people out of SLAM, because their features move. Here nothing was masked. Is this trajectory good enough to put objects into a map?**

Activity 6: Mask the People, and the Objects

Run SLAM again on the same tracks, with the mask of Lab 02 on the people. First segment the sequence with DeepLabV3, as in Lab 02 (its Activity 8), then add the mask:

```
pixi run semantic-segmentation --method DEEPLABV3 --tum $S --save labels/walking
pixi run slam --tum $S --settings settings/TUM3.yaml --sensor rgbd \
    --tracks tracks/walking_xyz --semantic-mask labels/walking \
    --mask-classes person --mask-dilation 10
```

The segmentation takes about a minute on a GPU. On a CPU add `--step 5`, as in Lab 02, or get the label maps from your group; while it runs, read the object lines of Activity 5 again. The mask comes from your function `mask_from_labels()` of Lab 02 (its Part 3). If that still returns `None`, the run says that no frame was masked. The run ends as in Activity 5, after one more line which says how much was masked:

```
Semantic mask: F of 827 frames masked; P% of the pixels masked on average
```

Fill in rows 1 and 2 of Table 4, from the last lines of Activity 5's run and of this one.

	Objects	Static	Moving	ATE rmse (m)	Monitor (m)
1. Tracks, no mask (Activity 5)					
2. Tracks, the person mask					
3. Lab 04's detections, the mask		–	–		–

Table 4: The SLAM runs of Part 3 on `walking_xyz`. Static and Moving: the static and the moving tracks of the `Objects:` line. Monitor: the distance within which 90% of the points of the monitor (`tv`) with the most tracks lie.

- **How did the mask change the trajectory error, and the objects? Both runs lifted the same tracks: what made the difference?**
- **Count the monitors and the chairs in the images of the sequence. Which of the two lists is right?**
- **Which tracks became objects, and which were left as moving? Are any of the objects people? If so, how can a person be static? Should the rule know that people move?**

- **Find the monitor with the most tracks in both runs' lists. Its tracks are the same: did its cloud change with the mask, and why?**

Lab 04 lifted each detection on its own, without tracks. Run its two commands on this sequence, with the same mask, and compare its objects with these:

```
pixi run -e semantics object-detection --method YOLO --tum $S \
    --settings settings/TUM3.yaml --save detections/walking_xyz --step 5
pixi run slam --tum $S --settings settings/TUM3.yaml --sensor rgbd \
    --detections detections/walking_xyz --semantic-mask labels/walking \
    --mask-classes person --mask-dilation 10
```

The run ends with Lab 04's object line, `Objects: N from D detections ...`**: write its objects and the trajectory error into row 3 of Table 4.**

- **Which gives the cleaner list of objects: the detections or the tracks? Where do the extra objects of the other list come from? Look at the people: what does Lab 04's merge do with a person who walks across the room? Lab 04 lifts only the detections scoring at least 0.5, in every fifth frame; the tracks keep boxes down to 0.1, in every frame.**
- **The last number of Lab 04's** `Objects:` **line counts the objects made of a single detection. What happens to a single false detection in the tracks' run?**
- **The map now holds points with labels (Lab 02), objects with classes, placed in the map (Lab 04), and tracks over time. What could a robot ask such a map that it could not ask before? What can it still not answer?**

Worked Example: Objects on a Street

The tracks of the worked example of Part 1, saved with `--save` **and lifted by a SLAM run of the same video, as in Activity 5 but with** `--video` **and without** `--sensor rgbd`**. The video has no depth images, so SLAM runs monocular: each box gets the median depth of the map points that SLAM tracks inside it, and the map has a scale of its own. The run says so, and gives its distances in map units:**

```
Tracks: 1513 boxes lifted in 990 frames, with the map points (monocular: the
map's own scale); 1688 boxes had too little depth
Monocular: the map has a scale of its own, so --static-radius and
--merge-radius are in its units, not in metres
Objects: 52 from 83 static tracks: car x43, truck x9; moving: 20 tracks (...)
  car: centre (-1.32, 0.03, 3.02) map units; 4 tracks (672, 651, 683, 717),
  107 boxes; 90 % of its points within 0.28 map units
  truck: centre (0.06, -0.16, 10.54) map units; 4 tracks (1846, 292, 1936,
  1966), 95 boxes; 90 % of its points within 0.17 map units
  ...
```

That is `BYTETRACK`**'s tracks. A second run on the same tracks gave 48 objects (car x37, truck x10, motorcycle x1).** `SORT`**'s tracks gave 49 and 52 objects in two runs (cars, trucks and two buses), from 86 and 89 static tracks. The video has no ground truth for the trajectory, so there is no** `ATE` **line. More than half of the boxes are not lifted: a box is lifted only if SLAM tracks enough map points inside it.**

- **The truck above is made of tracks 292 and 1846 among others. Track 292 was seen in frames 196 to 225 and track 1846 in frames 977 to 1020: the drive passes the same place twice. Which part of the system joined the two passes into one object: the tracker, or the map?**

- **The two radii are 0.30 and 0.35 map units. Compare them with the distances between the centres of the objects in the list. What would you need to know to set the radii in metres, as on `walking_xyz`?**
- **Two runs on the same tracks gave different numbers of objects. Where does the difference come from, when the tracks are the same?**

7 Extra Activities

These are optional.

Activity 7: Score Against Known Tracks

Watching is not a measurement. The MOTChallenge (Multiple Object Tracking) benchmark has sequences on which the box and the identity of every pedestrian are marked by hand in every frame. Download MOT16, its 2016 set, as `MOT16.zip` (1.9 GB) from its web site, `motchallenge.net`, and unpack it. Its `train/MOT16-05` is 837 frames (640 by 480 pixels, 14 frames per second, so about one minute) of a pavement, filmed by a camera which moves. The hand-marked tracks give 125 people in 6818 boxes. Use it only for this module's work, as its licence allows. Score the three trackers on it:

```
pixi run -e semantics object-tracking --tracker BYTETRACK \
    --evaluate path/to/MOT16/train/MOT16-05
pixi run -e semantics object-tracking --tracker BOTSORT \
    --evaluate path/to/MOT16/train/MOT16-05
pixi run -e semantics object-tracking --tracker SORT \
    --evaluate path/to/MOT16/train/MOT16-05
```

Each takes under a minute on a GPU, and one to three minutes on a CPU. Only the people are scored. The summary gives three scores:

- **MOTA, multiple object tracking accuracy, counts the errors (missed people, false boxes and identity switches) as a fraction of the true boxes, and subtracts it from 1. An identity switch is a swap of two people's identities, or a restart of one person under a new identity.**
- **IDF1, the identification F1 score, asks how much of each true track was covered by one identity.**
- **HOTA, higher order tracking accuracy, scores detection (DetA) and association (AssA) separately, and combines them.**

The last command's summary, `SORT`'s, on an Apple M4 Max (`mps`) was:

```
Summary: SORT on YOLO, MOT16-05 (837 frames; 125 people, 6818 true boxes):
MOTA 0.47, IDF1 0.58, HOTA 0.44 (DetA 0.44, AssA 0.44); 126 identity
switches, 2638 misses, 821 false boxes; 279 identities given to the people;
0.023 s per frame on mps (the tracker alone 0.3 ms); no second stage (boxes
of 0.25 or more); 1313 tracks started, 2572 matched boxes held back while
their track was on probation
```

Fill in Table 5 from the summary lines; its last row is the line above.

On a CPU the scores were the same. On an NVIDIA GPU (`cuda`) a few counts differ by one to five, probably because the detector's arithmetic differs a little there, but the scores agree to two decimals. *125 people* are the true tracks; *279 identities* are the tracker's for the same people, so at least 154 of them are extra.

	MOTA	IDF1	HOTA	DetA	AssA	Switches	Misses	False	Identities
`BYTETRACK`									
`BOTSORT`									
`BYTETRACK`, **2nd stage off**									
`SORT` (ours)	*0.47*	*0.58*	*0.44*	*0.44*	*0.44*	*126*	*2638*	*821*	*279*

Table 5: The scores on MOT16-05, for the people only. Switches: identity switches. False: false boxes. Identities: the identities given to the 125 people.

- **Which tracker scores higher on MOTA, and which on IDF1? Can one tracker win on one score and lose on another? Which kind of mistake moves each score?**
- **Run `BYTETRACK` with `--second-stage off`. Which counts move, and in which direction? What does that say about the low-score boxes which the second stage matches on this street?**
- **Which tracker gives the 125 people the most identities? From its rules for starting, reporting and deleting a track (Table 1), why? Does the higher number of identities show in MOTA, in IDF1, or in neither?**
- **The camera of this sequence moves. Does BoT-SORT's compensation show in the scores, as Activity 2 suggested it might?**
- **The sequence is marked in every frame. If it were marked only in every third frame, which mistakes would the scores miss?**

Activity 8: Your Own Clip

Film 20 to 30 seconds with your phone: one or two members of your group walking across the view and past each other, while you walk slowly with the camera. Film only members of your group who have agreed to be filmed, and nobody else. Keep the clip only on the lab machine or on your UCL OneDrive, and delete it at the end of term. Run SLAM on it first, as in Lab 01, to check that it tracks, then the tracker, with the settings file you made in Lab 01:

```
pixi run -e semantics object-tracking --tracker BYTETRACK --video my/clip.mp4 \
    --settings settings/MY_CAMERA.yaml
```

There is no ground truth, so count by eye, from the `Timeline`: how many identities each person got, and where the changes happened.

- **Did a number change when the two crossed, when one left the frame, or when you turned the camera? Which of these would a tracker that knew the camera's motion survive?**
- **Repeat with `BOTSORT`. Did its compensation help on your clip?**

Activity 9: A Street

Run the worked examples of Parts 1 and 3 yourself: track the KITTI 06 video of Lab 00, and lift the tracks with a monocular SLAM run:

```
pixi run -e semantics object-tracking --tracker BYTETRACK \
    --video data/videos/kitti06/video_color.mp4 \
    --settings settings/KITTI04-12.yaml --save tracks/kitti06
pixi run slam --video data/videos/kitti06/video_color.mp4 \
    --settings settings/KITTI04-12.yaml --tracks tracks/kitti06
```

Without a depth image, each box gets the median depth of the map points that SLAM tracks inside it, in the map's own scale. The run says so.

- **How long does a parked car keep its identity as the camera drives past? What ends the track: the car leaving the frame, or a change of identity?**
- **Do the parked cars collapse to points? The two radii are now in the map's units, not in metres. How many objects does that make, and why?**

Activity 10: Tracking by Prompt

Run the tracker on the detections of `YOLOE`, which detects the classes you name (Lab 04). Lab 04's Setting Up downloaded the model and its text encoder, so nothing more is downloaded:

```
pixi run -e semantics object-tracking --tracker BYTETRACK --method YOLOE \
    --prompt "monitor, keyboard, person" --tum $S --settings settings/TUM3.yaml
```

- **In Lab 04, the word `monitor` did not find the monitors on the desk. Does it find this sequence's monitors?**
- **Does a prompted class keep its identity as well as a COCO class? Where does a monitor's track break?**